COMPSCI 3MI3 Assignment 1

Academic integrity and assignment verification

Per the course outline, this assignment will be worth 3% of your final grade. You are encouraged to discuss these problems and potential solutions with your classmates, and you are welcome to ask about them during office hours. Your submitted solutions must be entirely your own work, perhaps informed by discussions you have had with your peers, the instructional team, or generative AI tools used in accordance with university policies and the course outline.

As part of the assessment process, students may be selected for a brief follow-up discussion regarding their submissions. These discussions are intended to verify authorship and understanding of the submitted work. Every student should expect to participate in at least one such discussion during the term.

Following a discussion, the instructional team may adjust an assignment grade based on the student's demonstrated understanding of the assessed material. Students who can clearly explain concepts that were not fully demonstrated in their written submission may receive additional credit. Conversely, if a student is unable to demonstrate an understanding consistent with the submitted work, this may result in a charge of academic dishonesty and the application of penalties as outlined in the McMaster Academic Integrity Policy.

The purpose of these discussions is both to support learning and to help ensure the academic integrity of course assessments.

Submission instructions

You should submit your assignment solutions through Crowdmark no later than 11:59PM on Sunday, October 18th.

Questions

All three of these questions consider the language *arith* as defined in Pierce Chapter 3 and in class. They also consider a new function, *esses: arith -> Nat,* which counts the number of times the character *'s'* appears in a term. For example:

- *esses(0) = 0*
- *esses(false) = 1*
- *esses(if iszero 0 then succ 0 else false) = 4*

- *esses(pred succ 0) = 1*

1. Write down a recursive definition for the function esses. (You do not need to prove that your function works as expected.)
2. Let *size(t)* be the number of nodes appearing in the abstract syntax tree for *t*, as seen in Pierce Chapter 3 and in class. Prove (by structural induction) that *esses(t)* is bounded above by *size(t)*. I.e. $esses(t) \leq size(t)$
3. In Haskell, we can implement the language arith as a data type via the declaration:

```
data ArithTerm = TmTrue | TmFalse | TmZero
| TmSsucc ArithTerm | TmPred ArithTerm | TmIsZero ArithTerm
| TmIfThenElse ArithTerm ArithTerm ArithTerm
```

Write down code which implements the function *esses* in Haskell. Your code should start with the line:

```
esses :: ArithTerm -> Int
```